Лабораторна робота №5 WTF-форми

Мета роботи	• Побудова форм за допомогою Flask-WTF • Використання макросів для розмітки полів форми • Виведення flash-повідомлень про успішні/неуспішні дії та помилки під час обробки форми • Дотримання патерну "Post/Redirect/Get" для керування вхідними данними.

Кожне завдання виконувати поетапно, зберігаючи працездатність застосунку після кожної модифікації та змін у коді. Кожен етап розроблення чи вдосконалення застосунку потрібно документувати за допомогою Git-комітів із зрозумілим повідомленням, що відображає виконану роботу, і скріншоти зміненого інтерфейсу, який підтверджує результат виконаної роботи.

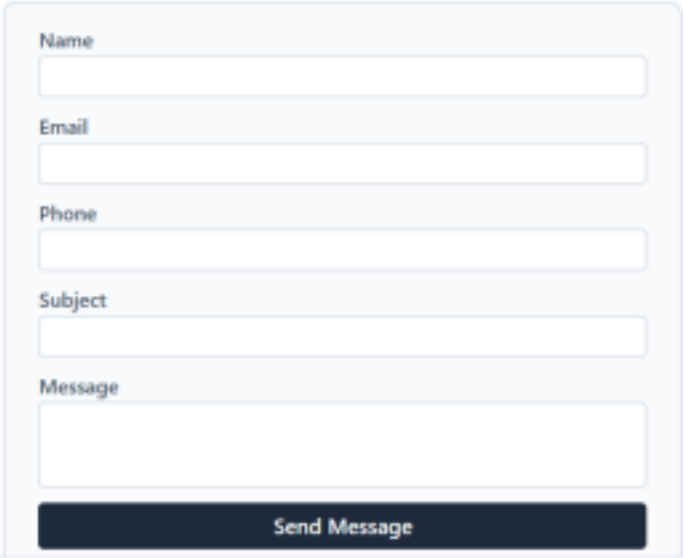
Приклад: `git commit -m "feat: add Flask-WTF registration form"`

Завдання 1 (2 бали)

Модифікувати існуючу контактну форму, використовуючи WTF-форми.

```
@app.route('/contact', methods=['GET', 'POST'])
def contact():
        #Імплементувати логіку відображення/обробки форми
 #сам клас форми описати в окремому файлі forms.py, скористатись імпортом
```

Контактна форма повинна містити відповідні текстові поля, випадаючий список, область для розширеного повідомлення та кнопку відправки.



Вимоги до полів	
name	Обов'язкове, довжина: min=4, max=10
email	Обов'язкове, почта
phone	Формат: +380?????????
subject	Зробити тип поля як SelectField. Тут має бути вибір із деякого випадаючого списку (задайте список самі).
message	Обов'язкове, довжина до 500 символів

Технічні вимоги:

1. Клас форми має бути винесений в окремий модуль forms.py 2. Забезпечити валідацію полів згідно вимог та захист від CSRF. 3. Дану форму у шаблоні сформувати за допомогою макросу
   відображення поля, в якому повинний бути блок з виводом можливих помилок у полі.
4. Добавити підтримку flash-повідомлення з категоріями (якщо нема).
5. При відправленні форми дані з неї мають:
   - дописуватись у деякий log-файл (можна використати модуль logging або loguru).
   - Видаватись flash-повідомлення про успішну/неуспішну відправку і записування даних користувача.
6. Дотримуватись техніки *Post/Redirect /Get mode*.

Приклад можливого підключення flash-повідомлень

```
{% for cat, msg in get_flashed_messages(True) %}
<ul class="list-group">
<!--      <li class="list-group-item list-group-item-info">{{msg}}</li> -->
    <li class="list-group-item list-group-item-{{cat}}">{{msg}}</li>
</ul>
{%endfor %}
```

Приклад можливого опису макроса та його використання для виводу форми:

templates > <> _render_field.html > ...

```
1   {% macro render_field(field) %}
2
3    <dt>{{ field.label }}
4
5   {% if field.errors %}
6   <dd>{{ field(style="background-color: red;", **kwargs) | safe }}
7           <ul>
8           {% for error in field.errors %}
9           <li  class="list-group-item list-group-item-danger"> {{error}} </li>
10          {% endfor %}
11          </ul>
12  {% else %}
13  <dd>{{ field(**kwargs) | safe }} </dd>
14  <br>
15  {% endif %}
16
17  {% endmacro %}
```

Завдання 2 (3 бали)

Модифікувати і вдосконалити блюпринт products з використанням TinyDB для зберігання даних про продукти та Flask-WTF для створення й валідації форм.

1. Підключення та налаштування TinyDB

Встановити бібліотеку `tinydb`, підключити її до Flask-застосунку та налаштувати зберігання даних у JSON-файлі `data/products.json`. Реалізувати ініціалізацію сховища та таблиці зберігання товарів.

2. Структура товару

Кожен продукт повинен містити такі поля:

- `id` — унікальний ідентифікатор;
- `name` — назва товару (мінімум 3 символи);
- `content` — опис товару (текстова область 5 × 40);
- `price` — додатна дробова ціна з точністю до двох знаків після коми;
- `category` — категорія товару (із заданого списку);
- `quantity` — кількість одиниць (невід'ємне ціле число);
- `image` — шлях до зображення товару;
- `created_at` — дата й час створення, встановлюються автоматично.

3. Функціональність CRUD

Реалізувати/модифікувати такі операції:

Операція	**Маршрут**	**Основні вимоги**
Додавання (Create)	`GET/POST /product/add`	Форма WTF із полями `name`, `content`, `price`, `category`, `quantity`, `image`. Валідація даних, автоматичне встановлення `id` і `created_at`, збереження в TinyDB, PRG-паттерн і flash-повідомлення.
Перегляд (Read)	`GET /product`	Відображення списку товарів із назвою, категорією, ціною, кількістю одиниць, зображенням і часом додавання.
Перегляд товару	`GET /product/<int:id>`	Відображення повної інформації про товар, включно з описом. Обробка випадку, коли товар не знайдено.

Редагування (Update)	GET/POST /product/<int:id>/edit	Форма WTF з поточними даними товару, валідація полів, можливість замінити зображення або залишити попереднє. Збереження змін у TinyDB без зміни id і created_at.
Видалення (Delete)	POST /product/<int:id>/delete	видалення товару через POST-запит /product/<int:id>/delete із підтвердженням дії.

4. Зображення

Для роботи з файлами використовувати FileField із WTForms та відповідні валідатори. Передбачити:

- підтримку форматів PNG, JPG і JPEG;
- обмеження максимального розміру завантаження;
- безпечне формування імен файлів, наприклад за допомогою UUID;
- зберігання файлів у каталозі блюпринта static/uploads/;
- збереження відносного шляху до файлу в TinyDB;
- відображення зображень у списку товарів і на сторінці окремого товару;
- коректну обробку відсутнього зображення.

5. Зберігання даних

Зберігати інформацію про товари в JSON-файлі за допомогою TinyDB. Забезпечити автоматичне створення ідентифікаторів, збереження даних між запусками застосунку та коректне виконання операцій додавання, перегляду, редагування й видалення.

6. Тести

Операції CRUD покрити модульними тестами